AN AUTOMATED RESUME GRADING SYSTEM USING NLP AND ML

Project Report Submitted in Partial Fulfilment of the Requirements for the Degree of

Bachelor of Technology

in

Computer Science and Business Systems

by

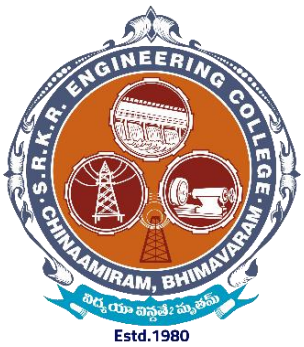
Jada Suresh (Reg. No.: 22B91A5722)

Chamakuri Venkata Ramachandrudu (Reg. No.: 22B91A5708)

Ganji Abhilash (Reg. No.: 22B91A5716)

Under the Supervision of

DR. P. RAVI KIRAN VARMA
Head of the Department



Department of Information Technology

SRKR Engineering College (A)

SRKR Marg, China Amiram, Bhimavaram - 534204

April, 2026

D E C L A R A T I O N

I hereby declare that the Project Report entitled "AN AUTOMATED RESUME GRADING SYSTEM USING NLP AND ML" which is being submitted to the SRKR Engineering College (A), China Amiram, Bhimavaram, in partial fulfillment of the requirements for the award of the Degree of Bachelor of Technology in Computer Science and Business Systems is a Bonafide report of the research work carried out by us. The material contained in this Project Report has not been submitted to any University or Institution for the award of any degree.

Place: Bhimavaram
Date:

Jada Suresh
Roll No.: 22B91A5722
Chamakuri Venkata Ramachandrudu
Roll No.: 22B91A5708
Ganji Abhilash
Roll No.: 22B91A5716

Department of Information Technology,
SRKR Engineering College (A),
SRKR Marg, China Amiram,
Bhimavaram-534204

C E R T I F I C A T E

This is to certify that the work contained in this report entitled "AN AUTOMATED RESUME GRADING SYSTEM USING NLP AND ML" is submitted by the group members Mr. Chamakuri Venkata Ramachandrudu (Reg. No.: 22B91A5708), Mr. Ganji Abhilash (Reg. No.: 22B91A5716) and Mr. Jada Suresh (Roll No.: 22B91A5722) to the Department of Information Technology, SRKR Engineering College, as the record of research work carried out by them. This report is accepted as the Project Report submission in partial fulfilment of the requirements for the award of the degree of Bachelor of Technology.

Dr. P. Ravi Kiran Varma
Department of Information Technology
Project Supervisor

Dr. P. Ravi Kiran Varma
Department of Information Technology
Head of the Department

ACKNOWLEDGEMENT

The victorious completion of this project would be incomplete without greeting those who made it possible and whose guidance and encouragement made efforts that led to the success.

The thesis presented here is the work accomplished under the enviable and scholarly guidance of "**Dr. Ravi Kiran Varma P**", Head of Information Technology Department. We are grateful towards the indomitable support. We express deep gratitude for his inspiring remarks and helping this project.

We take this opportunity to express our sincere thanks to "**Dr. Ravi Kiran Varma P** ", Head of the Department, Information Technology for his support and encouragement.

We express our sincere thanks to "**Prof. K V Murali Krishnam Raju**", PRINCIPAL, S.R.K.R. Engineering College, Bhimavaram for giving us this opportunity for the successful completion of this project.

Last but not least, we also express our sincere thanks to the other teaching and nonteaching staff for their support in accomplishing this project.

Name	Register No
JADA SURESH	(22B91A5722)
CHAMAKURI VENKATA RAMACHANDRUDU	(22B91A5708)
GANJI ABHILASH	(22B91A5716)

ABSTRACT

Human Resource (HR) department faces significant challenges in screening resumes for scheduling interviews for a given job role. In modern recruitment scenarios, organizations often receive hundreds or even thousands of applications for a single position, making manual shortlisting of candidates extremely time-consuming, labor-intensive, and prone to inconsistencies. The manual evaluation process depends heavily on the subjective judgment of recruiters, which can introduce bias and lead to the rejection of potentially qualified candidates. Therefore, there is a strong need for an automated, efficient, and reliable system that can assist HR professionals in identifying the most suitable candidates. Machine Learning (ML) and Natural Language Processing (NLP) techniques have emerged as powerful tools to address this problem by enabling automated resume screening and grading. These techniques can process large volumes of unstructured resume data, extract meaningful information, and classify candidates based on predefined criteria. However, one of the major challenges in designing such systems is identifying the most relevant features that accurately represent a candidate's skills, experience, and overall competence. Simply relying on basic attributes such as skills or keywords is not sufficient to capture the complete profile of a candidate.

In this work, a dataset of resumes is collected from both successful and unsuccessful pass-out students. These resumes are carefully evaluated and graded by alumni and experienced faculty members to ensure high-quality and reliable labeling. The resumes, which are in PDF format and often contain unstructured and inconsistent content, are processed using NLP techniques and regular expressions to extract important features. These features include skills, number of projects, certifications, GitHub repositories and their quality, participation in hackathons, coding problem-solving activity, professional memberships, LinkedIn presence, and CGPA. The feature set is then used to train and evaluate multiple ML algorithms, including Logistic Regression, Decision Tree Classifier, XGBoost Classifier, Linear Support Vector Machine (SVM), and KNN. The experimental results demonstrate that the XGBoost, achieving an accuracy of approximately 93%.

LIST OF FIGURES

FIGURE NO	FIGURE NAME	PAGE NO
3.1	System Architecture	10
4.1	Confusion Matrix of XGBoost Classifier	18
4.2	ROC curves of various classifiers under evaluation.	19
4.3	Precision–Recall–F1 Comparison	19
4.4	Proposed System interface showing resume upload module	20
4.5	System result interface showing grade prediction	21

LIST OF TABLES

TABLE NO	TABLE NAME	PAGE NO
4.1	Resume grading accuracies produced by various ML algorithms	17

TABLE OF CONTENTS

S. NO	CHAPTER	PAGE NO.
	ABSTRACT	i
	LIST OF FIGURES	ii
	LIST OF TABLES	ii
1	**INTRODUCTION**	**1 - 4**
1.1	Problem Statement	1
1.2	Objectives of the Project	1
1.3	Scope of the Study	2
1.4	Importance of the Work	2
1.5	Reason for Choosing the Topic	3
2	**LITERATURE REVIEW**	**5-7**
2.1	Overview of Related Work	5
2.2	Comparative Analysis of Existing Approaches	6
2.3	Research Gap Identification	7
3	**EXPERIMENTAL METHODOLOGY**	**9-16**
3.1	Proposed System	9
3.2	System Architecture	10
3.3	Module Description	11

3.4	Implementation Methodology	13
3.5	Proposed Algorithms	14
4	**TEST RESULTS**	**17-21**
4.1	System Performance Evolution Metrics	17
4.2	Confusion Matrix Analysis	17
4.3	ROC Curve Analysis	18
4.4	Precision–Recall–F1 Comparison	19
4.5	System Interface and Result Visualization	20
5	**DISCUSSION OF RESULTS**	**23-24**
5.1	Comparative Analysis of Machine Learning Models	23
5.2	Error Analysis and Model Limitations	24
6	**CONCLUSION**	**25-26**
6.1	Scope for the Future Work	25
	REFERENCES	**27**
ANNEXURE 1	**SOURCE CODE**	**29-48**
	PRINTOUT OF PAPER ACCEPTANCE	**49**
ANNEXURE 1	**RESEARCH PAPER**	**50-55**

CHAPTER 1

INTRODUCTION

1.1 Problem Statement

The increasing number of job applications has made manual resume screening time-consuming, inefficient, and prone to bias. Recruiters often need to evaluate a large number of resumes, which leads to inconsistent decisions and the possibility of overlooking qualified candidates. Existing automated systems mainly rely on keyword-based skill matching [2]. Although these methods improve processing speed, they fail to capture the overall competency and real-world abilities of candidates. Resumes with relevant skills may be misclassified if the information is presented in different formats or wording.

Moreover, important factors such as projects, certifications, GitHub activity, hackathon participation, coding practice, professional memberships, LinkedIn presence, and academic performance (CGPA) are often not properly considered. These features are essential for evaluating a candidate's practical knowledge and technical strength. Therefore, there is a need for an automated and reliable resume grading system that goes beyond simple keyword matching. By using Natural Language Processing (NLP) and Machine Learning (ML) techniques, the system can analyze multiple features and provide a more accurate and consistent evaluation of resumes. Therefore, there is a need for an automated, objective, and scalable resume grading system that goes beyond simple keyword matching. Such a system should leverage Natural Language Processing (NLP) and Machine Learning (ML) techniques to extract, analyze, and evaluate multiple quantitative and qualitative features, thereby enabling a more accurate, consistent, and fair assessment of candidates in modern recruitment processes.

1.2 Objectives of the Project

The main objectives of this project are:

- To automate resume grading using ML and NLP.
- To identify meaningful resume features beyond skill matching.
- To compare multiple ML classifiers for resume quality grading.
- To determine the most accurate ML model for grading resumes

1.3 Scope of the Study

The scope of this work is centered on academic and entry-level recruitment scenarios, with a primary focus on evaluating student resumes. The system is designed to process resumes in PDF format and automatically assign grades ranging from A to D based on predefined quality indicators. These indicators include multiple quantitative and qualitative features such as skills, projects, certifications, GitHub repository quality, hackathon participation, coding problem-solving activity, professional memberships, LinkedIn presence, and academic performance (CGPA). The dataset utilized in this study consists of resumes collected from a single institution from 2025 and 2026 batch, ensuring consistency in labeling and evaluation criteria. However, this controlled dataset limits the model's ability to generalize effectively across diverse industries, domains, and resume formats encountered in real-world recruitment environments.

The proposed system is intended to assist HR professionals by automating the initial stages of resume screening and grading, thereby reducing manual workload and improving processing efficiency. It enables faster shortlisting of candidates by providing a standardized and objective evaluation based on measurable attributes. However, the system is not designed to replace human decision-making; rather, it functions as a decision-support tool that complements the recruiter's expertise. Final hiring decisions still rely on human judgment, interviews, and additional evaluation criteria beyond resume analysis. Furthermore, the system is capable of handling unstructured resume data through the use of Natural Language Processing (NLP) techniques and rule-based feature extraction methods, ensuring that relevant information is consistently captured despite variations in formatting and presentation. While the current implementation demonstrates strong performance, its effectiveness is influenced by the size, diversity, and quality of the training dataset. Therefore, for large-scale or industry-level deployment, the model requires further training on more diverse datasets, continuous validation, and potential integration with real-time recruitment platforms. Future extensions may include domain-specific customization, adaptive learning mechanisms, and enhanced feature engineering to improve scalability and robustness.

1.4 Importance of the Work

The significance of this work lies in enhancing the efficiency, accuracy, and fairness of the recruitment process through automated resume screening. Conventional approaches

predominantly rely on manual evaluation or basic keyword matching techniques, which are often time-consuming, inconsistent, and prone to subjective bias. Such limitations can lead to unreliable candidate assessment and the potential exclusion of qualified applicants. To address these challenges, the proposed system integrates Natural Language Processing (NLP) and Machine Learning (ML) techniques to enable intelligent and automated analysis of resumes.

Unlike traditional systems, the proposed approach considers a comprehensive set of features, including number of projects, certifications, GitHub repositories and their quality, participation in hackathons, coding problem-solving activity, professional memberships, LinkedIn presence, and academic performance (CGPA). By incorporating both quantitative and qualitative attributes, the system provides a more holistic evaluation of a candidate's profile, moving beyond simple keyword-based filtering. Furthermore, the automation of the screening process significantly reduces the workload on HR professionals, leading to faster processing of large volumes of applications while maintaining consistency in evaluation. The use of data-driven decision-making minimizes human bias and ensures objective assessment based on measurable criteria. Additionally, the system is scalable and can be effectively deployed in real-world recruitment environments to handle increasing application volumes. Overall, the proposed system contributes to improving the reliability, transparency, and efficiency of candidate selection, making it a valuable solution for modern recruitment systems.

1.5 Reason for Choosing the Topic

The motivation for selecting this topic arises from the increasing challenges in the recruitment process, where organizations receive a large volume of resumes for a single job role. Manual screening of these resumes is not only time-consuming but also prone to inconsistencies and human bias, making it difficult to identify the most suitable candidates efficiently. Existing automated systems mainly rely on keyword matching techniques, which fail to capture the overall competence and practical abilities of candidates. Such approaches often overlook important aspects like number of projects, certifications, GitHub repositories and their quality, participation in hackathons, coding problem-solving activity, professional memberships, LinkedIn presence, and academic performance (CGPA).

This limitation motivated the development of an automated resume grading system using Natural Language Processing (NLP) and Machine Learning (ML) techniques. The proposed

system aims to provide a faster, more accurate, and unbiased evaluation by considering multiple relevant factors such as skills, projects, certifications, and practical experience, thereby improving the overall effectiveness of the recruitment process.

CHAPTER 2

LITERATURE REVIEW

The rapid growth of online recruitment has created a need for intelligent, data-driven resume screening systems. This literature review focuses on research in resume analysis, feature extraction, and machine learning techniques to support the development of an automated resume grading system.

2.1 Overview of Related Work

Vishaline et al. [1] used various Machine Learning (ML) models like Support Vector Machine (SVM), Naive Bayes, and XG Boost for resume ranking, they have found XG Boost is most suitable model with reported AUC-ROC value of 0.89.Tejaswini et al. [2] developed a resume ranking system using Natural Language Processing (NLP) text processing, pruning, feature extraction, and further used cosine similarity calculations to identify the top rated resumes. They used machine learning model called K-Nearest Neighbors (KNN) for final selection and ranking of resumes. They have reported 92% accuracy on ranking and 86% accuracy of parsing. They have collected fifty resumes from Kaggle. Reddy et al. [3] used PyPDF2 library to parsing the resumes and extract relevant text, they used Google Gemini Api for semantic analysis and resume ranking. Roy et al. [4] used different machine learning (ML) models like Random Forest, Multinomial Naïve Bayes, Logistic Regression, Linear Support Vector Machine (SVM) Classifier for classifying resumes based on departments, they found Linear SVM classifier is most suitable model with accuracy 78.53%. Then K-Nearest Neighbors (KNN) is used for Resume Ranking. Chennupalle et al. [5] integrate compiler design principles for lexical analysis with Machine Learning (ML) models, This system efficiently parses resume data into feature vectors and Word2Vec embeddings by combining machine learning models with compiler design approaches for lexical analysis. After five models were compared, Random Forest was found to be the best, categorising individuals into 14 different job titles with an accuracy of 98.29%. Tayal et al. [6] developed a system by employing NLP techniques like named entity recognition and part-of-speech tagging, coupled with ML classifiers such as K-Nearest Neighbors (KNN) and Support Vector Machines (SVM). They Compared Machine Learning (ML) models like KNN, SVM, One vs Rest, where they found KNN as the most effective model with accuracy 98.44%. Padmaja et al. [7] used NLP algorithms such as SBERT and cosine similarity for identifying resumes that are most relevant to a specific job description. 5

Surendiran et al. [8] used different Machine Learning (ML) models like Decision Tree, Random For-est, K-Nearest Neighbors (KNN), Support Vector Machine (SVM), they have found Random Forest model as the most effective with a 91.38% accuracy. They used dataset of 3,446 resumes across 48 categories. Onukwugha et al. [9] used Multinomial Naive Bayes and K-Nearest Neighbors (KNN) algorithms to predict resume effectiveness and categorize them. They found that the KNN method worked best with accuracy 87%. They used dataset of resumes from Kaggle. Bharadwaj et al. [10] developed a system utilizing Long Short-Term Memory (LSTM) neural networks and NLP to categorize resumes according to skills into various job options. They used total of 962 resumes from Kaggle. Mamta et al. [11] developed an automated resume screening application using Machine Learning (ML) and NLP to categorize resumes. They used different ML models where K-Nearest Neighbors (KNN) found to be best with accuracy 98.44%. Surya et al. [12] used Support Vector Machines (SVM), Logistic Regression, Random Forest and K-Nearest Neighbors (KNN) Machine Learning models for resume ranking, out of all they found SVM as Best one with accuracy 96%. Gopalakrishna and Varadharajan. [13] used Ensemble Voting Classifier com-posed of six individual models like Naïve Bayes, Multinomial Naïve Bayes, Bernoulli Naïve Bayes, Linear Support Vector Machines (SVM), Logistic Regression, and K-Nearest Neighbors (KNN). Suhas HE and Manjunath AE. [14] used Named Entity Recognition (NER) combined with word2vec and cosine similarity to extract and analyze resume data. The accuracy obtained was 79.8%.

2.2 Comparative Analysis of Existing Approaches

Existing resume screening systems mainly use machine learning (ML) and natural language processing (NLP) techniques. Many researchers applied models like Naive Bayes, SVM, and XGBoost for ranking resumes, where XGBoost showed better performance [1]. Some systems use NLP and cosine similarity to compare resumes with job descriptions and rank candidates effectively [2]. Other approaches use models like Logistic Regression, Random Forest, and SVM for classification, but their accuracy is moderate [4]. Several systems focus on extracting information from resumes using parsing techniques. Tools like PyPDF and regular expressions are used to convert unstructured resumes into structured data [3][5]. Some works also use NLP methods like Named Entity Recognition (NER) and part-of-speech tagging to improve feature extraction and classification accuracy [6][11]. Advanced approaches use semantic techniques like SBERT and word embeddings to understand the meaning of resume content and improve

matching with job roles [7][14]. Deep learning models like LSTM provide better understanding of context but require large datasets and more computation [10].

However, most existing systems focus only on basic features like skills and experience. They do not properly evaluate real-world coding ability. Even when GitHub is considered, it is usually treated as a simple feature instead of analyzing repository quality. This limits the system's effectiveness.In this paper, a more advanced approach is used by including features like number of projects, certifications, GitHub repositories, and their quality. Multiple ML models are compared, and XGBoost gives the best performance.

2.3 Research Gap Identification

Most existing automated resume screening and ranking systems largely depend on skill-based keyword matching, where resumes are evaluated primarily based on the presence and frequency of predefined skill terms[2][7]. While a few systems consider basic indicators such as the number of projects, they fail to assess the quality, complexity, or real-world relevance of those projects. Critical features such as GitHub repository quality, coding problem-solving activity, hackathon participation, certifications, professional memberships, and academic performance (CGPA) are either ignored or treated superficially. As a result, these systems provide an incomplete and biased evaluation of candidates, often favoring resumes with well-written skill sections rather than those with genuine technical competence and practical exposure. This limitation reduces the effectiveness and fairness of automated resume screening.

CHAPTER 3

EXPERIMENTAL METHODOLOGY

3.1 Proposed System

The proposed system is an automated resume grading framework designed to objectively evaluate and categorize resumes using Natural Language Processing (NLP) and Machine Learning (ML) techniques. The primary goal of this system is to assist Human Resource (HR) teams in efficiently screening large volumes of resumes while ensuring consistency, fairness, and accuracy in the evaluation process. Unlike traditional methods that rely heavily on manual effort or simple keyword-based filtering, this system provides a more comprehensive and intelligent approach to candidate assessment. The system accepts resumes in PDF format as input, which is a commonly used format in recruitment processes. Initially, the resumes undergo preprocessing, where textual content and embedded hyperlinks are extracted using PDF parsing libraries. In cases where resumes are scanned or image-based, Optical Character Recognition (OCR) is applied to convert the content into machine-readable text. This ensures that the system can handle different types of resume formats effectively. The extracted text is further cleaned by removing noise such as special characters, unnecessary symbols, and redundant whitespace to improve the quality of data for further processing.

Once the preprocessing is complete, the system performs feature extraction using a combination of NLP techniques, regular expressions, and external APIs. Important attributes such as skills, number of projects, certifications, GitHub repositories and their quality, hackathon participation, coding problem-solving activity, professional memberships, LinkedIn presence, and academic performance (CGPA) are identified and extracted. This multi-dimensional feature extraction approach allows the system to capture both qualitative and quantitative aspects of a candidate's profile, providing a more holistic evaluation compared to traditional systems that focus only on keywords.The extracted features are then transformed into a structured format suitable for machine learning models. Textual features, particularly skills, are converted into numerical representations using the Term Frequency–Inverse Document Frequency (TF-IDF) technique, which assigns importance to each skill based on its frequency and relevance across all resumes. Numerical features such as project count, certifications, and CGPA are normalized to ensure uniform scaling and to prevent bias during model training. All features are then combined into a unified feature vector representing each resume.

This feature vector is used as input for multiple machine learning classifiers, including Logistic Regression, Decision Tree, Support Vector Machine (SVM), K-Nearest Neighbors (KNN), and XGBoost. These models are trained and evaluated using standard performance metrics such as accuracy, precision, recall, and F1-score. Based on comparative analysis, the XGBoost classifier is identified as the best-performing model due to its ability to handle complex feature relationships, reduce overfitting, and deliver high predictive accuracy.Finally, the trained model predicts a grade for each resume, categorizing it into one of four classes: A, B, C, or D. This grading system enables HR teams to quickly identify high-quality candidates and make informed decisions during the initial screening phase. Overall, the proposed system offers a scalable, efficient, and unbiased solution for resume evaluation, significantly reducing manual workload while improving the reliability and effectiveness of the recruitment process.

3.2 System Architecture

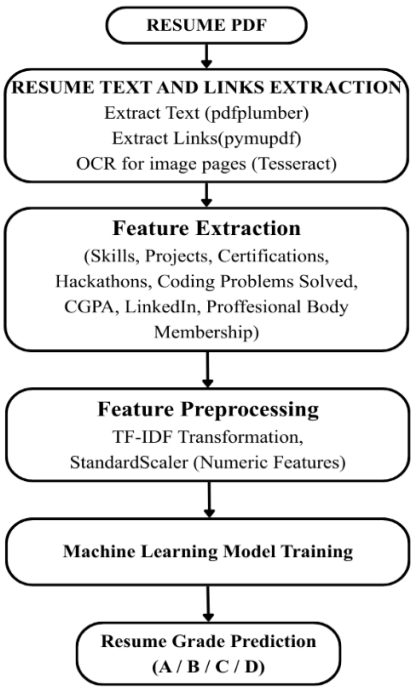


Fig 3.1. Architecture Diagram for proposed algorithm

Fig 3.1. illustrates the workflow of the proposed system, which follows a multi-stage architecture for automated resume grading. The process begins with extracting text and hyperlinks from resumes provided in PDF format. In cases where the resume is image-based or scanned, Optical Character Recognition (OCR) is applied to ensure complete data extraction.

The extracted content is then processed using a combination of regular expressions and Natural Language Processing (NLP) techniques to identify and extract key attributes. These attributes include skills, number of projects, certifications, GitHub repositories and their quality, hackathon participation, coding problem-solving activity, professional profiles, and academic performance (CGPA). This step converts unstructured resume data into a structured format suitable for analysis. The processed features are further refined and transformed into numerical representations, enabling them to be used as input for machine learning models. These features are then fed into a trained classification model, where multiple algorithms are evaluated to determine the most effective one. The model analyzes the relationships between different attributes and predicts a final grade for each resume, categorizing it into A, B, C, or D.

Overall, the system provides an automated, scalable, and efficient solution for resume evaluation, reducing manual effort while ensuring consistent and objective grading based on multiple performance indicators.

3.3 Module Description

The proposed automated resume grading system follows a modular architecture designed to ensure scalability, maintainability, and efficient processing of large volumes of resumes. Each module performs a well-defined task and interacts with adjacent modules to form a seamless end-to-end pipeline. This design enables independent development, testing, and optimization of individual components while maintaining overall system integrity.

3.3.1 Resume Input Interface Module

This module serves as the entry point of the system, enabling users to upload resumes through a web-based interface. The system accepts resumes in PDF format and performs validation checks to ensure compatibility and data integrity. Input validation mechanisms include file type verification, size constraints, and error handling for corrupted or incomplete files. This ensures that only valid inputs are forwarded to subsequent stages, thereby reducing processing errors and improving system reliability.

3.3.2 Resume Parsing and Preprocessing Module

This module is responsible for converting unstructured resume data into structured textual content. PDF parsing libraries such as PdfPlumber and PyMuPDF are used to extract text and

embedded hyperlinks. For image-based or scanned resumes, Optical Character Recognition (OCR) is applied to recover textual information. The extracted text undergoes preprocessing steps including normalization, tokenization, and removal of irrelevant symbols to ensure consistency across diverse resume formats. This module plays a critical role in bridging the gap between unstructured input data and structured feature representation required for machine learning.

3.3.3 Feature Extraction Module

This module extracts relevant attributes that contribute to resume evaluation. These features include both qualitative and quantitative aspects such as skills, number of projects, certifications, GitHub repositories, hackathon participation, coding activity, LinkedIn profile, professional memberships, and CGPA.A hybrid extraction approach is adopted, combining Natural Language Processing (NLP), rule-based methods, and semantic analysis. This ensures accurate identification of features even when resumes follow different structures or terminologies. The extracted features provide a comprehensive representation of a candidate's academic background, practical exposure, and technical competency.

3.3.4 GitHub Repository Scoring Module

This module evaluates the quality of a candidate's GitHub profile by analyzing repository-level metrics. Data is retrieved using the GitHub REST API, and multiple indicators such as stars, forks, contributors, pull requests, issue activity, and repository freshness are considered. A multi-criteria decision-making approach based on the Technique for Order Preference by Similarity to Ideal Solution (TOPSIS) is employed to compute a normalized GitHub score. This approach ensures that repositories are evaluated based on their relative closeness to ideal characteristics. The inclusion of this module enhances the system's ability to assess real-world coding proficiency beyond traditional resume attributes.

3.3.5 Feature Transformation and Representation Module

This module transforms extracted features into numerical representations suitable for machine learning models. Textual features, particularly skills, are vectorized using the Term Frequency–Inverse Document Frequency (TF-IDF) technique, which assigns higher importance to discriminative terms. Numerical features are normalized to ensure uniform scaling and prevent bias toward features with larger magnitudes. The transformed textual and numerical features

are then integrated into a unified feature vector. This standardized representation improves model learning efficiency and ensures balanced contribution from all features.

3.3.6 Machine Learning Classification Module

This module performs the core task of resume grading using supervised machine learning algorithms. Multiple models, including Logistic Regression, Decision Tree, Support Vector Machine (SVM), K-Nearest Neighbors (KNN), and XGBoost, are evaluated. Among these, XGBoost is selected as the optimal model due to its superior performance in handling complex, non-linear feature relationships. The model leverages gradient boosting techniques to iteratively improve prediction accuracy while incorporating regularization to prevent overfitting. The classifier assigns resumes into four categories: A, B, C, and D, representing varying levels of candidate quality.

3.3.7 Result Generation and Feedback Module

This module presents the final prediction results to the user through an interactive interface. The system displays the predicted grade along with a numerical score and provides actionable feedback for improvement.The results can also be stored for further analysis, enabling continuous monitoring and enhancement of system performance. This module ensures that the system is not only predictive but also assistive in improving resume quality.

3.4 Implementation Methodology

The implementation of the proposed automated resume grading system is carried out through a structured and sequential methodology to ensure reliability and accuracy. Initially, resumes are collected in PDF format and undergo preprocessing to handle variations in layout, formatting, and content structure. Textual content and embedded hyperlinks are extracted using PDF processing libraries, and in cases where resumes are scanned or image-based, Optical Character Recognition (OCR) is employed to convert images into machine-readable text. The extracted text is then cleaned by removing noise such as special characters, redundant whitespace, and irrelevant symbols. Natural Language Processing techniques are applied to segment the resume content into meaningful sections such as skills, projects, certifications, and academic details. Feature extraction is performed using a hybrid approach that combines rule-based methods, regular expressions, and semantic analysis. Skills are identified using both keyword matching and contextual understanding through sentence embeddings, ensuring robustness to synonym

usage. Numerical and categorical features such as project count, certifications, GitHub repositories, hackathon participation, coding problem-solving activity, professional memberships, LinkedIn presence, and CGPA are extracted and structured into tabular form. Textual features are transformed into numerical representations using the TF-IDF technique, while numerical features are normalized to avoid bias due to scale differences. The complete feature set is then divided into training and testing datasets. Multiple machine learning models are trained and evaluated using standard performance metrics. Comparative analysis is conducted to identify the most effective model for resume grading, which is then selected for final deployment.

3.5 Proposed Algorithms

Algorithm 3.1 Feature extraction and grading

Input: pdf resume

- Extract text T and links L from PDF.
- If T is empty → apply OCR.
- Extract skills S using hybrid NLP.
- Count projects P from project section.
- Count certifications C from certification section.
- Extract GitHub username U.
- Fetch repositories and compute GitHub score GS.
- Extract hackathons H and coding problems CP.
- Detect LinkedIn and memberships.
- Extract CGPA value.
- Construct feature vector X.
- Transform skills using TF-IDF

$$TFIDF(t,d) = TF(t,d) \cdot \log\left(\frac{N}{DF(t)}\right)$$

- Predict grade using trained XGBoost classifier.

Output: Return final grade G ∈ {A, B, C, D}.

The proposed algorithm takes a resume in PDF format as input and begins by extracting textual content and embedded links. If the resume does not contain readable text (such as scanned documents), an Optical Character Recognition (OCR) technique is applied to retrieve the content. The extracted text is then processed using hybrid NLP methods to identify important features such as skills, number of projects, certifications, GitHub profile, hackathon participation, coding problem-solving activity, LinkedIn presence, professional memberships, and CGPA.Once the GitHub username is identified, repository data is fetched and a GitHub score is calculated to evaluate the quality of technical contributions. All extracted attributes are combined to form a structured feature vector. The skills section is further transformed into numerical form using the TF-IDF technique to assign importance to each skill based on its frequency and relevance. Finally, the processed feature vector is given as input to a trained XGBoost classifier, which predicts the overall resume grade as A, B, C, or D.

Algorithm 3.2. GitHub repositories scoring algorithm

- Construct decision matrix for all n repositories using 7 metrics:

$$X = [x_{ij}], i = 1..n, j = 1..7$$

- Normalize criteria values using vector normalization:

$$r_{ij} = \frac{x_{ij}}{\sqrt{\sum_{k=1}^{n} x_{kj}^2}}$$

- Apply weights to get weighted matrix:

$$v_{ij} = w_j \cdot r_{ij}, \sum_{j=1}^{7} w_j = 1$$

- Calculate TOPSIS closeness using ideal best and worst

$$C_i = \frac{D_i^-}{D_i^+ + D_i^-}$$

Where,

$$D_i^+ = \sqrt{\sum_{j=1}^{7} (v_{ij} - A_j^+)^2},\ D_i^- = \sqrt{\sum_{j=1}^{7} (v_{ij} - A_j^-)^2}$$

- Final user score (0–100) as average repo closeness:

$$score(U) = \frac{100}{n} \sum_{i=1}^{n} C_i$$

The algorithm 3.2. shows the workflow of scoring GitHub repositories, In this TOPSIS

technique is employed in our GitHub scoring algorithm to rank the repositories on various engineered quality factors like stars, forks, contributors, pull requests, issue health, freshness, and releases. A decision matrix is formed for each user, where each repository is described by these factors. The TOPSIS technique normalizes the values and assigns weights to the factors based on their relative importance [16]. The best and worst repository solutions are also determined. The distance of each repository from the best and worst solutions is calculated, and a closeness index is derived, which measures the relative closeness of a repository to the best engineered project. The average TOPSIS closeness index of all the repositories is then normalized to get a single GitHub repositories score between 0 and 100 for the user.

CHAPTER 4

TEST RESULTS

The performance of the proposed intrusion detection system was evaluated using standard classification metrics such as Accuracy, Precision, Recall, F1-score, ROC curve, and Confusion Matrix. These metrics help in understanding how effectively the system detects attacks and classifies them into different categories.

4. 1 System Performance Evolution Metrics

Table 4.1. Resume grading accuracies produced by various ML algorithms

ML Model	Precision	Recall	F1 score	Accuracy
Logistic Regression	0.80	0.60	0.65	0.74
Decision Tree Classifier	0.74	0.73	0.73	0.79
XG Boost Classifier	**0.92**	**0.84**	**0.86**	**0.93**
Linear SVM	0.57	0.61	0.59	0.79
K-Nearest Neighbors (KNN)	0.58	0.55	0.55	0.74

Table 4.1. presents the key performance metrics of various machine learning models considered in this study. Among all the models, the XGBoost classifier achieved the highest accuracy of approximately 93%. This superior performance is mainly due to its ensemble boosting nature, where multiple weak decision trees are combined to form a strong predictive model. XGBoost is also highly effective in capturing complex relationships between features and reducing overfitting through regularization techniques. Therefore, among all the evaluated models, XGBoost is found to be the most suitable for this work.

4. 2 Confusion Matrix Analysis

The confusion matrix for the results obtained using the XGBoost classifier is presented in Fig 4.1. It shows that out of 42 test resumes, 39 resumes are correctly classified, resulting in an overall accuracy of approximately 93%. This indicates that the model performs effectively in distinguishing between different resume quality categories.

A closer analysis of class-wise performance reveals that the model achieves 100% accuracy in predicting Class A and Class C resumes, indicating strong learning for clearly distinguishable patterns in high-quality and mid-level resumes. However, the performance is comparatively lower for Class B and Class D, with accuracies of 88% and 50%, respectively. This variation may be due to overlapping features or limited training samples for these classes, making them harder to distinguish.

Overall, the confusion matrix highlights that while the XGBoost classifier demonstrates high overall accuracy and strong performance for certain classes, there is still scope for improvement in handling borderline or less-represented categories.

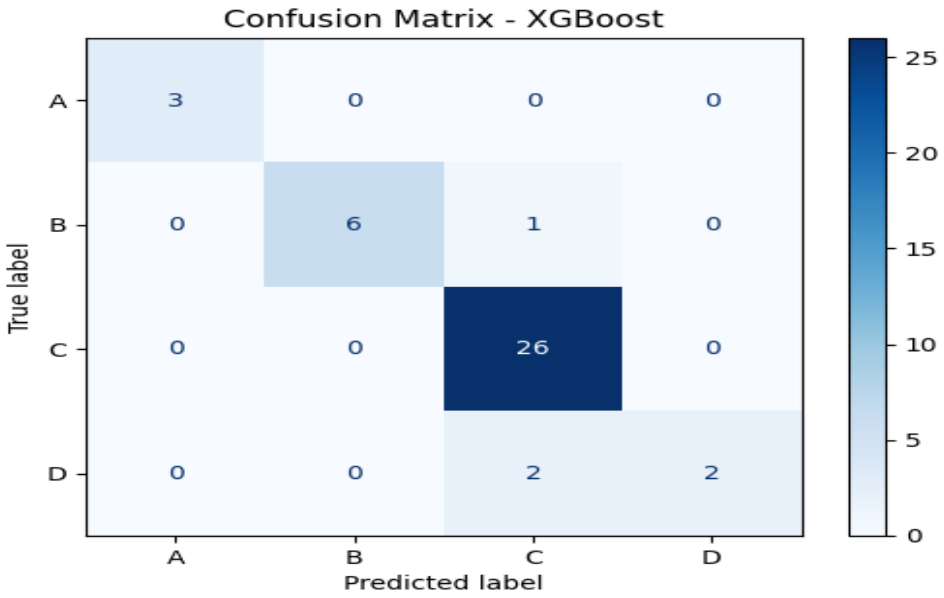


Fig 4.1. Confusion Matrix for XGBoost

4.3 ROC Curve Analysis

Fig 4.2. gives the graphical comparison of ROC curves of the classifiers performances under evaluation. Fig 4.2. clearly shows the performance of XGBoost over others. Among them, XGBoost reaches the highest AUC value, 0.91, thus providing the best performance among all models and a strong capacity to differentiate between positive and negative instances of resume quality. Logistic Regression performs competitively with an AUC of 0.84, while SVM and Decision Tree both have a middle discrimination capability for the two classes with AUC values of 0.85 and 0.74 respectively. In contrast, KNN has the worst AUC value 0.79.

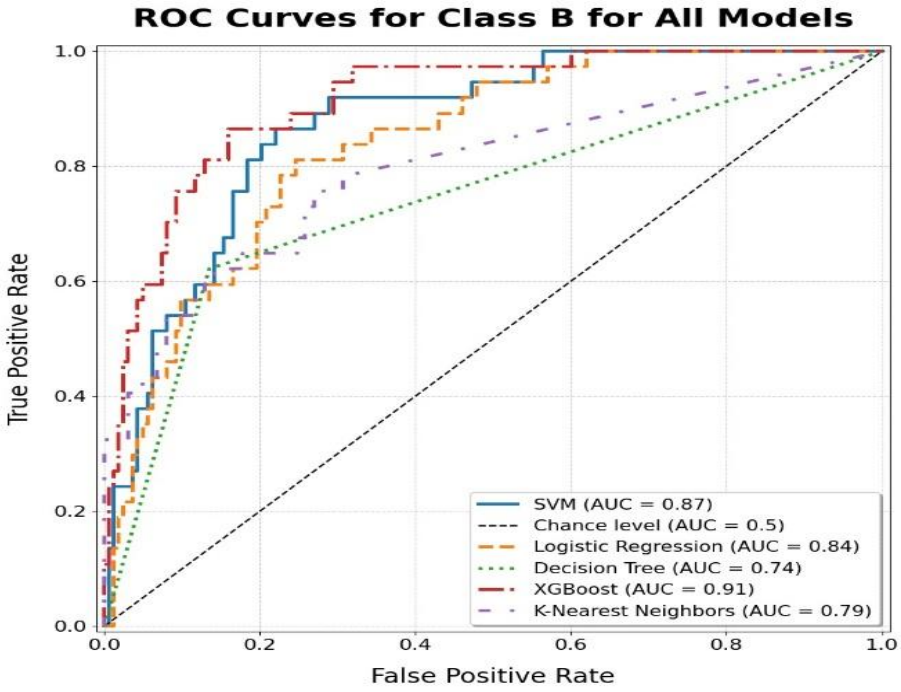


Fig 4.2. ROC curves of various classifiers under evaluation

4.4 Precision–Recall–F1 Comparison

As illustrated in Fig 4.3., the XGBoost model achieved the highest performance with a precision of approximately 0.86, recall of 0.82, and F1-score of 0.79, indicating its superior capability in accurately classifying resumes. In contrast, KNN exhibited the lowest performance with an F1-score of around 0.64, suggesting limitations in handling complex feature relationships.

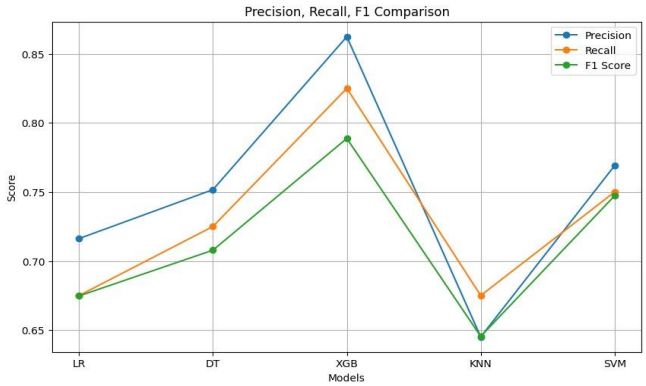


Fig 4.3. Precision, Recall, F1 Comparison

4.5 System Interface and Result Visualization

The graphical user interface of the proposed system is illustrated in Fig. 4.4 and Fig. 4.5. The system is implemented as a web-based application that enables users to upload resumes and obtain automated grading results using machine learning models. Fig. 4.4 presents the main interface of the system, which includes a resume upload module. Users can upload resumes through a drag-and-drop mechanism or by manually selecting a file. Upon submission, the system processes the resume using the backend pipeline, which includes PDF parsing, feature extraction, and classification using the trained XGBoost model.

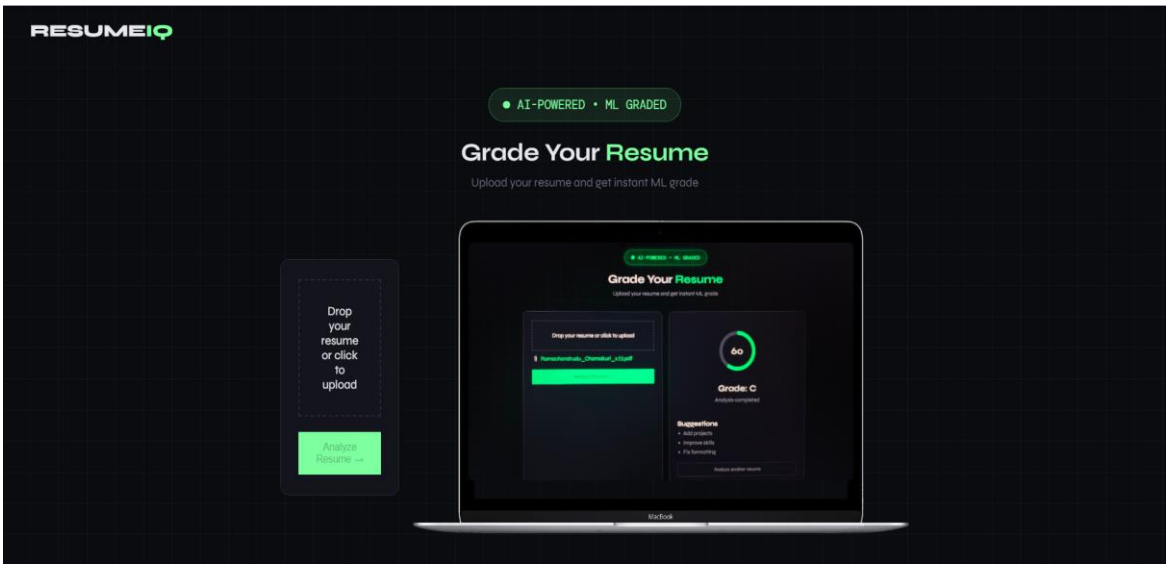


Fig. 4.4. Proposed System interface showing resume upload module

Fig. 4.5 shows the result visualization interface generated after processing the uploaded resume. The system displays the predicted grade (A–D) along with a numerical score and a graphical indicator for better interpretability. In addition, the interface provides actionable suggestions to improve resume quality, such as enhancing technical skills, adding projects, and improving formatting. The results are generated in near real-time, demonstrating the efficiency of the proposed system. The integration of prediction output with user-centric feedback ensures both usability and interpretability. Overall, the interface validates the practical applicability of the system for automated resume evaluation.

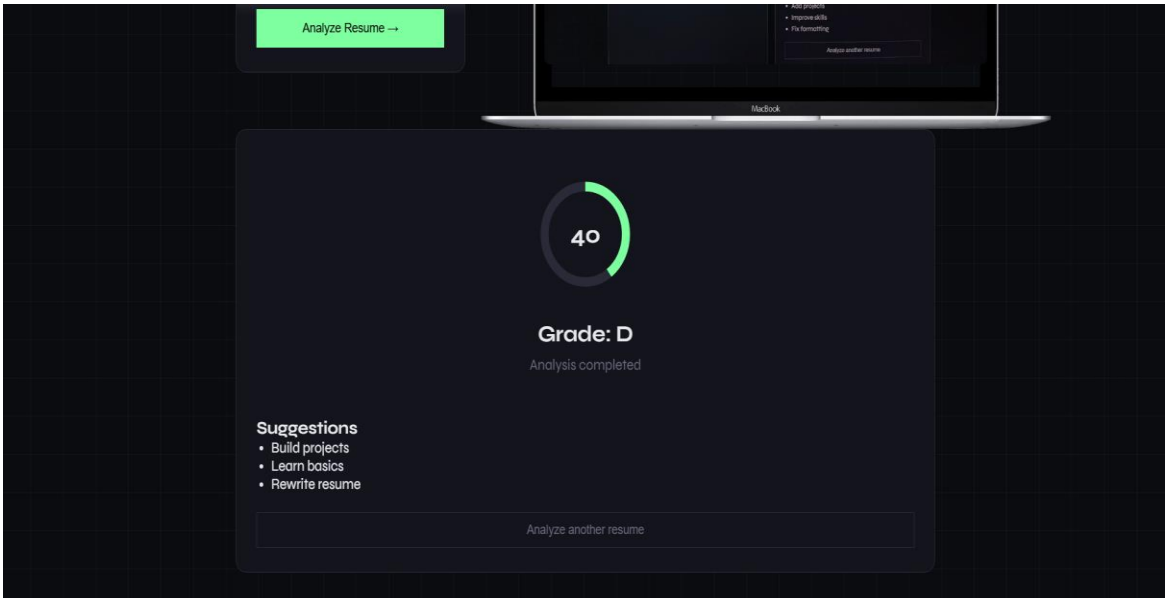


Fig. 4.5. System result interface showing grade prediction

CHAPTER 5

DISCUSSION OF RESULTS

5.1 Comparative Analysis of Machine Learning Models

The experimental evaluation reveals significant variation in the performance of the considered machine learning models. Traditional classifiers such as Logistic Regression and Decision Tree exhibit moderate performance, achieving accuracies of 74% and 79%, respectively. These models are inherently limited in their ability to capture complex non-linear relationships and interactions among features, which are prevalent in resume data. The Linear Support Vector Machine (SVM) achieves a comparable accuracy of 79%; however, its relatively low precision of 0.57 indicates a higher rate of false positives, thereby reducing its reliability in accurately identifying high-quality resumes. Likewise, the K-Nearest Neighbors (KNN) algorithm records the lowest performance, with an F1-score of 0.55, highlighting its sensitivity to high-dimensional feature spaces and its dependence on distance-based metrics, which become less effective as dimensionality increases.

In contrast, the XGBoost classifier demonstrates superior performance across all evaluation metrics, achieving an accuracy of 93%, precision of 0.92, recall of 0.84, and an F1-score of 0.86. The improved performance of XGBoost can be attributed to its gradient boosting framework, which iteratively enhances model performance by minimizing errors from previous iterations. Its ability to model complex feature interactions, along with built-in regularization mechanisms, helps in effectively reducing overfitting and improving generalization capability.

Furthermore, the higher recall value achieved by XGBoost indicates its effectiveness in correctly identifying relevant instances, while its high precision reflects a lower rate of false classifications. This balance between precision and recall is crucial in resume grading systems, where both accurate selection and minimization of misclassification are important. Additionally, the robustness of XGBoost in handling heterogeneous feature types, including both numerical and textual representations, contributes to its superior classification capability. Overall, the comparative analysis confirms that ensemble-based approaches, particularly XGBoost, are more suitable for this application due to their scalability, robustness, and ability to handle complex data distributions. These characteristics make XGBoost an effective choice for developing a reliable and efficient automated resume grading system.

5.2 Error Analysis and Model Limitations

Despite achieving high overall accuracy, the proposed model exhibits certain limitations that require further attention. The confusion matrix analysis reveals that while the model performs exceptionally well for Class A and Class C (100% accuracy), its performance decreases for Class B (88%) and significantly drops for Class D (50%). This variation indicates the presence of class imbalance and overlapping feature distributions among certain categories. Specifically, resumes belonging to borderline classes often share similar attributes, making it challenging for the model to distinguish between them accurately. Furthermore, the relatively small size of the test dataset (42 samples) may affect the reliability and generalizability of the results. A limited dataset increases the risk of overfitting and reduces the robustness of model evaluation.

To address these limitations, future improvements may include the application of data balancing techniques such as Synthetic Minority Over-sampling Technique (SMOTE), enhanced feature engineering strategies, and hyperparameter optimization methods. Expanding the dataset and incorporating additional relevant features can further improve classification performance and model generalization.

CHAPTER 6

CONCLUSION

In this paper, a resume grading system is developed using Machine Learning (ML) and Natural Language Processing (NLP) techniques to assist the HR team in efficiently evaluating and shortlisting candidates. A dataset of 200 resumes from the 2025 and 2026 batches of our college is collected and used for this study. These resumes are carefully graded by alumni and faculty experts to ensure accurate and reliable labeling for model training.The system extracts a wide range of important features from resumes using NLP techniques and regular expressions. These features include skill matching, number of projects, certifications, GitHub repositories and their quality, participation in hackathons, coding problem-solving activity, professional profiles, and academic performance measured through CGPA. This comprehensive feature extraction enables a more complete evaluation of a candidate's profile compared to traditional systems that rely only on keywords.

To evaluate the effectiveness of the proposed approach, multiple machine learning models such as Logistic Regression, Decision Tree Classifier, XGBoost Classifier, Linear Support Vector Machine (SVM), and K-Nearest Neighbors (KNN) are implemented and compared. Performance metrics such as accuracy, precision, recall, and F1-score are used for evaluation. Among all the models, the XGBoost classifier achieves the best performance with an accuracy of 0.93 and an F1-score of 0.88, due to its ability to handle complex feature relationships and reduce overfitting through ensemble learning.Overall, the proposed system reduces the manual workload of the HR team, improves the speed and consistency of resume evaluation, and provides a more reliable and fair assessment of candidates by considering both quantitative and qualitative attributes. This makes the system suitable for real-world recruitment applications and scalable for handling large volumes of resumes.

6.1 Scope for the Future Work

Although the proposed system demonstrates strong performance in automated resume grading, several enhancements can be explored to further improve its effectiveness and real-world applicability. One important direction is the expansion of the dataset to include resumes from diverse domains, institutions, and experience levels. A larger and more varied dataset will improve model generalization and reduce bias toward specific patterns observed in the current dataset. Another potential improvement is the integration of job description-based matching,

where resumes are evaluated relative to specific job requirements rather than general grading. This will enable more targeted and context-aware candidate selection.The system can also be enhanced by incorporating advanced deep learning models such as transformer-based architectures for better semantic understanding of resume content. This would improve the system's ability to interpret context, synonyms, and complex language structures.In addition, the inclusion of Explainable AI (XAI) techniques can improve transparency by providing detailed insights into feature contributions influencing the model's decisions. This will increase user trust and make the system more suitable for real-world deployment.

Further improvements may include real-time integration with recruitment platforms, automated feedback generation tailored to individual resumes, and continuous learning mechanisms that update the model based on new data. Overall, these enhancements will make the system more robust, scalable, and adaptable to evolving recruitment requirements.

REFERENCES

[1] R. Chinnasamy, M. Subramanian, S. V. Easwaramoorthy, and J. Cho, "Deep learning-driven methods for network-based intrusion detection systems: A systematic review," *ICT Express*, vol. 11, no. 1, pp. 181–215, 2025.

[2] H. Ali, M. Abbas, and E. Mehdi, "A comprehensive survey on intrusion detection systems with advances in machine learning, deep learning and emerging cybersecurity challenges," *Discover Artificial Intelligence*, vol. 5, p. 314, 2025.

[3] A. D. Vibhute, C. H. Patil, A. V. Mane, and K. V. Kale, "Towards detection of network anomalies using machine learning algorithms on the NSL-KDD benchmark datasets," *Procedia Computer Science*, vol. 233, pp. 960–969, 2024.

[4] J. Lansky, S. Ali, M. Mohammadi, M. K. Majeed, S. H. T. Karim, S. Rashidi, M. Hosseinzadeh, and A. M. Rahmani, "Deep learning-based intrusion detection systems: A systematic review," *IEEE Access*, vol. 9, pp. 101574–101599, 2021.

[5] H. C. Altunay and Z. Albayrak, "A hybrid CNN+LSTM-based intrusion detection system for industrial IoT networks," *Engineering Science and Technology, an International Journal*, vol. 38, 2023.

[6] B. Al-Omar and Z. Trabelsi, "Intrusion detection using attention-based CNN-LSTM model," in *Proc. Artificial Intelligence Applications and Innovations (AIAI)*, Springer, 2023, pp. 515–526.

[7] S. M. Kasongo, "A deep learning technique for intrusion detection system using a recurrent neural networks based framework," *Computer Communications*, vol. 199, pp. 113–125, 2023.

[8] N. Shone, T. N. Ngoc, V. D. Phai, and Q. Shi, "A deep learning approach to network intrusion detection," *IEEE Transactions on Emerging Topics in Computational Intelligence*, vol. 2, no. 1, pp. 41–50, 2018.

[9] I. Guyon and A. Elisseeff, "An introduction to variable and feature selection," *Journal of Machine Learning Research*, vol. 3, pp. 1157–1182, 2003.

[10] Y. Xin, L. Kong, Z. Liu, Y. Chen, Y. Li, H. Zhu, M. Gao, H. Hou, and C. Wang, "Machine

learning and deep learning methods for cybersecurity," *IEEE Access*, vol. 6, pp. 35365–35381, 2018.

[11] J. O. Mebawondu, O. D. Alowolodu, J. O. Mebawondu, and A. O. Adetunmbi, "Network intrusion detection system using supervised learning paradigm," *Scientific African*, vol. 9, p. e00497, 2020.

[12] N. Kshetri, "Transforming cybersecurity with agentic AI to combat emerging cyber threats," *Telecommunications Policy*, vol. 49, no. 6, 2025.

[13] N. Moustafa and J. Slay, "UNSW-NB15: A comprehensive data set for network intrusion detection systems," in *Proc. Military Communications and Information Systems Conference (MilCIS)*, Canberra, Australia, 2015, pp. 1–6, 2015.

[14] R. Mohammad, F. Saeed, A. A. Almazroi, F. S. Alsubaei, and A. A. Almazroi, "Enhancing intrusion detection systems using a deep learning and data augmentation approach," *Systems*, vol. 12, no. 3, p. 79, 2024.

[15] A. Kaissar, A. Bou Nassif, B. Soudan, and M. N. Injadat, "Enhancing CNN-based network intrusion detection through hyperparameter optimization," *Intelligent Systems with Applications*, vol. 26, p. 200528, 2025.

ANNEXURE 1

SOURCE CODE

ML SOURCE CODE:

```
# ================================
# IMPORT LIBRARIES
# ================================
import re
import pandas as pd
import numpy as np
import requests
import joblib
from datetime import datetime
from sklearn.model_selection import train_test_split
from sklearn.pipeline import Pipeline
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import StandardScaler, LabelEncoder
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics import classification_report
from xgboost import XGBClassifier
# ================================
# LOAD DATASET
# ================================
df = pd.read_csv("/content/DATASET_RESUME_GRADING.csv")
# Convert categorical features into numerical form
df["LinkedIn"] = df["LinkedIn"].map({"Yes": 1, "No": 0})
df["Professional Body Membership"] = df["Professional Body Membership"].apply(
    lambda x: 0 if pd.isna(x) else 1
)
```

```
# Separate features and target
X = df.drop(columns=["Name", "Grade"])
y = df["Grade"]
# Handle missing values
X["Skills"] = X["Skills"].fillna("")
X["LinkedIn"] = X["LinkedIn"].fillna(0)
# ================================
# DEFINE NUMERICAL FEATURES
# ================================
num_features = [
    "Projects",
    "Certifications",
    "GitHub Repos",
    "GitHub Repo Score",
    "Hackathons",
    "Coding Problems Solved",
    "LinkedIn",
    "Professional Body Membership",
    "CGPA"
]
# ================================
# CUSTOM TOKENIZER FOR SKILLS
# ================================
def skill_tokenizer(text):
    # Split skills based on comma, semicolon, or newline
    tokens = re.split(r'[,\n;]', text)
    # Clean tokens and convert to lowercase
    skills = [tok.strip().lower() for tok in tokens if tok.strip()]
    return skills
```

```
# ================================
# TF-IDF VECTORIZATION
# ================================
tfidf = TfidfVectorizer(
    stop_words='english',
    tokenizer=skill_tokenizer
)
# ================================
# PREPROCESSING PIPELINE
# ================================
preprocessor = ColumnTransformer(
    transformers=[
        ("skills", tfidf, "Skills"),            # Text feature transformation
        ("num", StandardScaler(), num_features) # Numerical feature scaling
    ]
)
# ================================
# LABEL ENCODING
# ================================
le = LabelEncoder()
y_encoded = le.fit_transform(y)
# ================================
# MODEL PIPELINE (XGBOOST)
# ================================
xgb_model = Pipeline([
    ("preprocess", preprocessor),
    ("clf", XGBClassifier(
        n_estimators=800,
        learning_rate=0.1,
```

```
        max_depth=6,
        subsample=0.85,
        colsample_bytree=0.85,
        reg_alpha=0.3,
        reg_lambda=1.2,
        gamma=0.2,
        eval_metric="mlogloss",
        random_state=42
    ))
])
# ================================
# TRAIN-TEST SPLIT
# ================================
X_train, X_test, y_train, y_test = train_test_split(
    X, y_encoded,
    test_size=0.2,
    random_state=42,
    stratify=y_encoded
)
# ================================
# MODEL TRAINING
# ================================
xgb_model.fit(X_train, y_train)
# ================================
```

FEATURES EXTRACTION SOURCE CODE:

```
import fitz
import pytesseract
import pdfplumber
```

```
import io
from PIL import Image
import re
import spacy
from difflib import SequenceMatcher

URL_PATTERN = re.compile(
    r"(https?://\S+|www\.\S+|github\.com/\S+|linkedin\.com/in/\S+)",
    re.IGNORECASE
)
TRAILING_JUNK = ")]}>.,;:\'"'
def clean_url(url):
    url = url.strip().strip(TRAILING_JUNK)
    if url.startswith("www."):
        url = "https://" + url
    if url.startswith("github.com"):
        url = "https://" + url
    if url.startswith("linkedin.com"):
        url = "https://" + url
    return url.lower()
def ocr_page(page):
    pix = page.get_pixmap(dpi=300)
    img = Image.open(io.BytesIO(pix.tobytes("png")))
    return pytesseract.image_to_string(img)
def extract_text_and_links(pdf_path):
    texts = []
    links = set()
    # -------- TEXT EXTRACTION --------
    with pdfplumber.open(pdf_path) as pdf:
        for page in pdf.pages:
            txt = page.extract_text(x_tolerance=2, y_tolerance=2)
            if txt and len(txt.strip()) > 30:
```

```
            texts.append(txt)
    doc = fitz.open(pdf_path)
    for page in doc:
        # OCR only if this page has little/no text
        if page.get_text().strip() == "":
            texts.append(ocr_page(page))
        # clickable links
        for link in page.get_links():
            uri = link.get("uri")
            if uri:
                links.add(clean_url(uri))
        # raw text URLs
        for url in URL_PATTERN.findall(page.get_text()):
            links.add(clean_url(url))
    doc.close()
    # -------- DEDUPLICATE TEXT BLOCKS --------
    final_text = "\n".join(dict.fromkeys(texts))
    return final_text.strip(), sorted(links)
#SKILL EXTRACTION
EXPLICIT_SKILLS = [
    "python", "java", "c", "c++", "javascript", "typescript", "go", "rust",
    "ruby", "php", "kotlin", "swift", "html", "css", "react", "angular",
    "vue", "next js", "bootstrap", "tailwind", "node js", "express",
    "django", "flask", "fastapi", "sql", "mysql",
    "postgresql", "mongodb", "redis", "aws", "azure", "gcp", "docker",
    "kubernetes", "git", "github", "postman", "swagger"
]
SKILL_SYNONYMS = {
    "js": "javascript", "py": "python", "postgres": "postgresql",
    "node": "node js", "tf": "tensorflow", "ml": "machine learning"
}
```

```
IMPLICIT_SKILL_CONCEPTS = [
    "machine learning", "deep learning", "data science",
    "natural language processing", "computer vision",
    "backend development", "frontend development",
     "cloud computing",
    "data structures and algorithms", "system design"
]
def extract_skills_nlp(text):
    detected_skills = {}
    text_lower = text.lower()
    text_clean = re.sub(r'[^a-z0-9\s+#]', ' ', text_lower)
    for skill in EXPLICIT_SKILLS:
        if re.search(rf"\b{re.escape(skill)}(\s+|$)", text_clean):
            detected_skills[skill] = 0.98
    for key, value in SKILL_SYNONYMS.items():
        if re.search(rf"\b{re.escape(key)}\b", text_clean):
            detected_skills[value] = max(detected_skills.get(value, 0), 0.95)
    try:
        doc = nlp(text_lower)
        sentences = [sent.text.strip() for sent in doc.sents if len(sent.text.strip()) > 20]
        if sentences:
            sent_embeddings = embedder.encode(sentences, convert_to_tensor=True)
            concept_embeddings = embedder.encode(IMPLICIT_SKILL_CONCEPTS,
convert_to_tensor=True)
            cosine_scores = util.cos_sim(concept_embeddings, sent_embeddings)
            for j, concept in enumerate(IMPLICIT_SKILL_CONCEPTS):
                max_score = float(cosine_scores[j].max())

                threshold = 0.80 if "development" in concept else 0.85
                if max_score >= threshold:
                    detected_skills[concept] = max(detected_skills.get(concept, 0), max_score)
    except Exception as e:
```

```
        print("NLP Error:", e)
    skills_sorted = sorted(detected_skills.keys(), key=lambda x: (-detected_skills[x], x))
    return ", ".join(skills_sorted)

#Projects Extraction
def extract_projects(text):
    """
    Final stable project title counter.
    Distinguishes titles from sub-headings.
    """
    if not text or len(text) < 80:
        return 0
    lines = [l.strip() for l in text.splitlines() if l.strip()]
    PROJECT_HEADERS = {
        "projects", "project", "academic projects",
        "technical projects", "key projects"
    }
    STOP_HEADERS = {
        "experience", "education", "skills",
        "certifications", "internship", "achievements"
    }
    ACTION_WORDS = {
        "developed","implemented","designed","built",
        "created","engineered","performed","enabled",
        "integrated","used","worked","applied",
        "analyzed","trained","deployed"
    }
    def looks_like_title(line):
        words = line.split()
        return (
            not line.startswith(("•","-","–")) and
            2 <= len(words) <= 8 and
```

```
        words[0].lower() not in ACTION_WORDS
    )
# locate Projects section
start = -1
for i, line in enumerate(lines):
    if line.lower() in PROJECT_HEADERS:
        start = i + 1
        break
if start == -1:
    return 0
titles = []
i = start
while i < len(lines):
    line = lines[i]
    low = line.lower()
    if low in STOP_HEADERS:
        break
    if looks_like_title(line):
        # must have at least one description line
        if i + 1 >= len(lines):
            i += 1
            continue
        next_line = lines[i + 1]
        # next line must be a description
        is_description = (
            next_line.startswith(("•","-","–")) or
            next_line.split()[0].lower() in ACTION_WORDS or
            len(next_line.split()) > 8 or
            ":" in next_line  # <-- CRITICAL FIX for Revathi
        )
        # next-next line must NOT be another title (fixes Chandu)
        is_followed_by_title = (
```

```
            i + 2 < len(lines) and
            looks_like_title(lines[i + 2])
        )
        if is_description and not is_followed_by_title:
            titles.append(low)
            i += 1
            continue
    i += 1
  return len(set(titles))
#Certificates Extraction
def extract_certifications_nlp(text):
  CERT_HEADERS = {
    "certification", "certifications",
    "certificate", "certificates"
  }
  STOP_HEADERS = {
    "projects", "experience", "education",
    "skills", "internship", "achievements"
  }
  EXCLUDE_TERMS = {
    "hackathon", "workshop", "participation",
    "organized", "event", "internship"
  }
  lines = [l.strip() for l in text.splitlines() if l.strip()]
  # ---- locate certification section ----
  start, end = -1, len(lines)
  for i, line in enumerate(lines):
    if line.lower() in CERT_HEADERS:
      start = i + 1
      break
  if start == -1:
    return 0
```

```
for j in range(start, len(lines)):
    if lines[j].lower() in STOP_HEADERS:
        end = j
        break
cert_lines = lines[start:end]
certifications = []
for line in cert_lines:
    low = line.lower()
    # hard excludes
    if any(x in low for x in EXCLUDE_TERMS):
        continue
    # too short = junk
    if len(low.split()) < 3:
        continue
    doc = nlp(line)
    # NLP sentence rejection
    # full sentences usually have subject + verb
    if any(tok.dep_ == "nsubj" for tok in doc):
        continue
    nouns = [t for t in doc if t.pos_ in ("NOUN", "PROPN")]

    # must be noun-dominant
    if len(nouns) < 2:
        continue
    clean = re.sub(r"\s+", " ", low)
    # deduplicate
    if not any(
        SequenceMatcher(None, clean, c).ratio() > 0.85
        for c in certifications
    ):
        certifications.append(clean)
return len(certifications)
```

```
#Hackathon Extraction
def extract_hackathons_nlp(text):
    doc = nlp(text.lower())
    count = 0
    for sent in doc.sents:
        if any(keyword in sent.text for keyword in ["hackathon", "datathon", "codefest", "ideathon","flowthon"]) and any(word in sent.text for word in ["participated", "won", "developed", "built"]):
            count += 1
    return count
#Coding problems Exraction
def extract_coding_problems(text: str) -> int:
    """
    Extract total number of coding problems solved.
    Uses a single pass to prevent double-counting the same text.
    """
    patterns = [
        r'solved\s+(\d+\+?)\s+problems',
        r'(\d+\+?)\s+coding\s+problems',
        r'(\d+\+?)\s+problems\s+solved',
        r'solved\s+(\d+\+?)',
    ]
    combined_pattern = '|'.join(patterns)
    total_count = 0
    for match in re.finditer(combined_pattern, text, re.IGNORECASE):
        for group in match.groups():
            if group:
                clean_num = group.replace('+', '')
                try:
                    total_count += int(clean_num)
                except ValueError:
                    continue
                break
```

```
    return total_count
#CGPA Extraction
def extract_cgpa(text):
    text = text.lower()
    patterns = [
        r'(?:cgpa|gpa|cumulative gpa|c\.g\.p\.a)\s*[:\-]?\s*(\d\.\d{1,2})\s*/\s*10',
        r'(?:cgpa|gpa|cumulative gpa|c\.g\.p\.a)\s*[:\-]?\s*(\d\.\d{1,2})',
        r'(\d\.\d{1,2})\s*(?:cgpa|gpa)',
        r'(\d\.\d{1,2})\s*(?:out of)\s*10'
        r"\(.*?cgpa\s*of\s*(\d{1,2}\.\d{1,2}).*?\)"
    ]
    for p in patterns:
        m = re.search(p, text)
        if m:
            val = float(m.group(1))
            if 0.0 < val <= 10.0 and val != 10.0:
                return val
    return 0.0

#GitHub Username Extraction
def extract_github_username(pdf_path):
    text, links = extract_text_and_links(pdf_path)
    # 1 raw text
    raw = re.findall(r"github\.com/([A-Za-z0-9_-]+)", text, re.IGNORECASE)
    if raw:
        return raw[0].lower()
    # 2 clickable
    for link in links:
        m = re.search(r"github\.com/([A-Za-z0-9_-]+)", link, re.IGNORECASE)
        if m:
            return m.group(1).lower()
    return None
```

```
GITHUB_TOKEN =
"github_pat_11B7TLOMQ0VyfguhsUKyg7_ozRTtAghb7W1nR8apnt0i7arWgzc6ibPxhjtstio
l6pI27D7JSAAgGsrmKf"

GITHUB_API = "https://api.github.com"

# ----------------------------

# Helper: GitHub GET with Token

# ----------------------------

def github_get(url):

    headers = {"Accept": "application/vnd.github+json"}

    if GITHUB_TOKEN:

        headers["Authorization"] = f"token {GITHUB_TOKEN}"

    r = requests.get(url, headers=headers)

    if r.status_code == 403:

        print(f"ERROR: GitHub API Rate Limit Hit or invalid token for URL: {url}")

        return None # Return None instead of raising an exception to allow graceful handling

    elif r.status_code != 200:

        print(f"ERROR: GitHub API request failed with status {r.status_code} for URL: {url} -
{r.text}")

        return None # Return None for other non-200 status codes


    return r.json()

# ----------------------------

# Extract Repo Metrics (7 Features)

# ----------------------------

def extract_repo_metrics(owner, repo):

    repo_url = f"{GITHUB_API}/repos/{owner}/{repo}"

    data = github_get(repo_url)

    if data is None:

        return None # Return None if primary repo data could not be fetched

    stars = data.get("stargazers_count", 0)

    forks = data.get("forks_count", 0)

    open_issues = data.get("open_issues_count", 0)
```

```
    # Freshness Score (0–1)
    pushed_at = data.get("pushed_at")
    days_since_update = (
        (datetime.datetime.now(datetime.timezone.utc) -
         datetime.datetime.strptime(pushed_at, "%Y-%m-
%dT%H:%M:%SZ").replace(tzinfo=datetime.timezone.utc)).days # Ensure parsed datetime is
timezone-aware
        if pushed_at else 9999
    )
    freshness = max(0, 1 - min(days_since_update, 365) / 365)
    # Contributors (limit heavy call)
    contrib_url = f"{repo_url}/contributors?per_page=30"
    contrib_data = github_get(contrib_url)
    contributors = len(contrib_data) if isinstance(contrib_data, list) else 1
    #  Pull Requests (limit)
    pr_url = f"{repo_url}/pulls?state=all&per_page=30"
    pr_data = github_get(pr_url)
    prs = len(pr_data) if isinstance(pr_data, list) else 0

    #  Issue Health (closed ratio)
    closed_url =
f"{GITHUB_API}/search/issues?q=repo:{owner}/{repo}+type:issue+state:closed"
    closed_data = github_get(closed_url)
    closed_issues = closed_data.get("total_count", 0) if isinstance(closed_data, dict) else 0
    total_issues = closed_issues + open_issues
    issue_health = closed_issues / total_issues if total_issues > 0 else 0
    # Releases
    release_url = f"{repo_url}/releases?per_page=10"
    release_data = github_get(release_url)
    releases = len(release_data) if isinstance(release_data, list) else 0
    return [stars, forks, contributors, prs, issue_health, freshness, releases]
```

```
# ----------------------------
#  TOPSIS Algorithm
# ----------------------------
def topsis(matrix, weights):
    matrix = np.array(matrix, dtype=float)
    #  Normalize matrix
    norms = np.sqrt((matrix ** 2).sum(axis=0))
    norms[norms == 0] = 1 # Avoid division by zero
    norm_matrix = matrix / norms
    #  Apply weights
    weighted_matrix = norm_matrix * weights
    #  Student Resume Ideal Benchmarks
    ideal_best_raw = np.array([
        3,   # Stars (student max)
        2,    # Forks
        1,    # Contributors
        1,    # PRs
        1,  # Issue health
        1,  # Freshness
        1     # Releases
    ])
    ideal_worst_raw = np.array([
        0,    # Stars
        0,    # Forks
        0,    # Contributors (minimum 1)
        0,    # PRs
        0,  # Issue health
        0,  # Freshness
        0     # Releases
    ])
    #  Normalize ideals the SAME way
    ideal_best = (ideal_best_raw / norms) * weights
```

```
    ideal_worst = (ideal_worst_raw / norms) * weights
    #  Distances
    dist_best = np.sqrt(((weighted_matrix - ideal_best) ** 2).sum(axis=1))
    dist_worst = np.sqrt(((weighted_matrix - ideal_worst) ** 2).sum(axis=1))
    #  TOPSIS Closeness Score
    # Add a small epsilon to avoid division by zero if both dist_best and dist_worst are zero
    epsilon = 1e-10
    scores = dist_worst / (dist_best + dist_worst + epsilon)
    return scores
# ----------------------------
#  GitHub Profile Score (0–100)
# ----------------------------
def github_score(username):
    repos_url = f"{GITHUB_API}/users/{username}/repos?per_page=30"
    repos = github_get(repos_url)

    if repos is None or not isinstance(repos, list) or len(repos) == 0:
        print(f"INFO: No repositories found for user {username} or API call failed.")
        return 0, 0
    decision_matrix = []
    #  Limit repos (avoid rate limit)
    for repo in repos[:15]:
        owner = repo["owner"]["login"]
        name = repo["name"]
        try:
            metrics = extract_repo_metrics(owner, name)
            if metrics is not None: # Only append if metrics were successfully extracted
                decision_matrix.append(metrics)
            else:
                print(f"WARNING: Skipping repo {owner}/{name} due to failed metric
extraction.")
        except Exception as e:
```

```
            print(f"ERROR: An unexpected error occurred while extracting metrics for {owner}/{name}: {e}")
            continue

    if len(decision_matrix) == 0:
        print("INFO: No valid repository metrics collected after processing. Returning 0, 0.")
        return 0, 0
    # RepoQuester Weights (Engineering Resume Focus)
    weights = np.array([
        0.25,  # Stars
        0.10,  # Forks
        0.15,  # Contributors
        0.15,  # PRs
        0.15,  # Issue Health
        0.20,  # Freshness
        0.10   # Releases
    ])

    weights = weights / weights.sum()
    #  Run TOPSIS
    repo_scores = topsis(decision_matrix, weights)
    # Profile-Level Aggregation
    profile_score = np.mean(repo_scores)
    final_score = round(profile_score * 100, 2)
    return len(repo_scores), final_score
#LinkedIN Detection
def detect_linkedin(text, links):
    """
    Detect LinkedIn presence in resume text or embedded links.
    Returns 1 if LinkedIn profile is found, else 0.
    """
    text_lower = text.lower()
```

```
    if any("linkedin.com" in l for l in links):
        return 1
    if "linkedin" in text_lower:
        return 1
    return 0
#Professional Body Membership Detection
def detect_professional_membership(text):
    """
    Detect memberships in professional bodies like IEEE, ISTE, CSI.
    Returns 1 if found, else 0.
    """
    text_lower = text.lower()
    bodies_patterns = [
        r"\biste\b",
        r"\bieee\b",
        r"\bcsi\b"
    ]
    for pattern in bodies_patterns:
        if re.search(pattern, text_lower):
            return 1
    return 0

def build_feature_row_nlp(pdf_path):
    text, links = extract_text_and_links(pdf_path)
    github_user = extract_github_username(pdf_path)
    repos, repo_score = github_score(github_user)
    return pd.DataFrame({
        "Skills": [extract_skills_nlp(text)],
        "Projects": [extract_projects(text)],
        "Certifications": [extract_certifications_nlp(text)],
        "GitHub Repos": [repos],
        "GitHub Repo Score": [repo_score],
```

```
        "Hackathons": [extract_hackathons_nlp(text)],
        "Coding Problems Solved": [extract_coding_problems(text)],
        "LinkedIn": [detect_linkedin(text, links)],
        "Professional Body Membership": [detect_professional_membership(text)],
        "CGPA": [extract_cgpa(text)]
    })

# Prediction
def predict_resume_grade(pdf_path):
    model_filename = "/content/resume_grading_xgb_mode.pkl"
    model_path = f"{model_filename}"
    model = joblib.load(model_path)
    X = build_feature_row_nlp(pdf_path)
    predicted_grade = model.predict(X)[0]
    if model_filename == "/content/resume_grading_xgb_mode.pkl":
        if predicted_grade == 0:
            predicted_grade = "A"
        elif predicted_grade == 1:
            predicted_grade = "B"
        elif predicted_grade == 2:
            predicted_grade = "C"
        else:
            predicted_grade = "D"
    return predicted_grade
grade = predict_resume_grade("/content/CHANDU_RES (1) (1) (1) (1).pdf")
print("Predicted Grade:", grade)
```

PRINTOUT OF PAPER ACCEPTANCE



Acceptance Letter

Paper Title: **An automated resume grading system using NLP and ML**

Acceptance id: **ICICI-064**

Authors: **Ravi Kiran Varma Penmatsa,Venkata Ramachandrudu Chamakuri,Abhilash Ganji,Suresh Jada**

Dear Authors,

On behalf of the Conference committee, we would like to congratulate you on your article to the **ICICI 2026** IEEE Conference, which will be held from **10-12, June 2026** at **S.E.A College of Engineering and Technology**, Bengaluru, Karnataka, India. You have been selected to deliver your presentation at the 4th International Conference on Inventive Computing and Informatics - ICICI 2026.

As a result of the review and results, we are pleased inform that you can now submit the camera-ready paper for inclusion into the ICICI proceedings. We appreciate if you could send the final version of your research paper at your earliest convenience, in order to ensure the timely publication. When submitting your final paper, please highlight the changes made according to the review comments.

Dr. Usha Desai
Organizing Secretary
ICICI 2026

Proceedings by

IEEE

ANNEXURE 2

RESEARCH PAPER

An automated resume grading system using NLP and ML

Ravi Kiran Varma Penmatsa
Department of Information Technology
Sagi Rama Krishnam Raju Engineering College
Bhimavaram, India
ravikiranvarmap@gmail.com

Venkata Ramachandrudu Chamakuri
Department of Information Technology
Sagi Rama Krishnam Raju Engineering College
Bhimavaram, India
chanduchamakuri@gmail.com

Abhilash Ganji
Department of Information Technology
Sagi Rama Krishnam Raju Engineering College
Bhimavaram, India
ganjiabhi1001@gmail.com

Suresh Jada
Department of Information Technology
Sagi Rama Krishnam Raju Engineering College
Bhimavaram, India
sureshjada2005@gmail.com

***Abstract*— The Human Resource (HR) department faces a challenge in screening resumes for scheduling interviews for a job role. Shortlisting the best resumes manually is nearly impossible when the number of applications is large. Machine Learning (ML) and Natural Language Processing (NLP) methods can help in selecting the best resumes. However, identifying the most appropriate attributes of the resume and judging its quality is challenging. In this paper, several resumes of successful and unsuccessful pass-out students are collected and graded using alumni and faculty experts. NLP and regular expressions are used to extract various useful features from the pdf resume document. A few ML tools like Logistic Regression, Decision Tree Classifier, XGBoost Classifier, Linear Support Vector Machine (SVM), and K-Nearest Neighbors (KNN) algorithms are considered for evaluating the data. XGBoost Classifier is found to be the best performer with an accuracy of 93% approximately.**

Keywords—NLP, Machine Learning, Resume Grading, Resume classification.

I. INTRODUCTION

Recruitment is resource intensive process in any company. Companies usually receive thousands of resumes for a job role. Human Resource (HR) department faces a challenge in screening resumes for scheduling interviews to a job role. Grading resumes manually is near to impossible when the applications are large in numbers. Machine Learning (ML) and Natural Language Processing (NLP) methods can help in grading resumes. However, most existing systems focus only on limited features such skills matching, ignoring features like number of projects, number of certifications, number of GitHub repositories, Quality of GitHub repositories, number of hackathons participated, number of coding problems solved, CGPA etc. In this work, we used NLP methods for creating dataset

from resumes. Around 200 resumes collected from 2025 and 2026 pass out batches of our college and graded using alumni and faculty experts. K-Nearest Neighbors (KNN), Logistic Regression, Linear Support Vector Machine (SVM), Decision Tree Classifier, XGBoost Classifier algorithms are considered to evaluate the data.

Vishaline et al. [1] used various Machine Learning (ML) models like Naive Bayes, Support Vector Machine (SVM), and XG Boost for resume ranking, they have found XG Boost is most suitable model with reported AUC-ROC value of 0.89. Tejaswini et al. [2] detailed a resume ranking architecture using Natural Language Processing (NLP) text processing, pruning, feature extraction, and further used cosine similarity calculations to identify the top rated resumes. They used KNN for final selection and ranking of resumes. They have reported 92% accuracy on ranking and 86% accuracy of parsing. They have collected fifty resumes from Kaggle.

Reddy et al. [3] used PyPDF2 library to parsing the resumes and extract relevant text, they used Google Gemini Api for semantic analysis and resume ranking. Roy et al. [4] used different ML tools including Linear Support Vector Machine (SVM) Classifier, Logistic Regression, Random Forest, Naïve Bayes, for classifying resumes based on departments, they found Linear SVM classifier is

most suitable model with accuracy 78.53%. Then K-Nearest Neighbors (KNN) is used for Resume Ranking. Chennupalle et al. [5] integrate compiler design principles for lexical analysis with Machine Learning (ML) models, This system efficiently parses resume data into feature vectors and Word2Vec embeddings by combining machine learning models with compiler design approaches for lexical analysis. After five models were compared, Random Forest was found to be the best, categorising individuals into 14 different job titles with an accuracy of 98.29%.

Tayal et al. [6] proposed a system employing NLP tools including KNN-SVM along with part-of-speech tagging, named entity recognition. They Compared Machine Learning (ML) models like KNN, SVM, One vs Rest, where they found KNN as the most effective model with accuracy 98.44%. Padmaja et al. [7] used NLP algorithms such as SBERT and cosine similarity for identifying resumes that are most relevant to a specific job description. Surendiran et al. [8] used different Machine Learning (ML) models like Decision Tree, Random Forest, KNN, Support Vector Machine (SVM), they have found Random Forest model as the most effective with a 91.38% accuracy. They used dataset of 3,446 resumes across 48 categories. Onukwugha et al. [9] used Multinomial Naive Bayes and K-Nearest Neighbors (KNN) algorithms to predict resume effectiveness and categorize them. They found that the KNN method worked best with accuracy 87%. They used dataset of resumes from Kaggle.

Bharadwaj et al. [10] developed a system utilizing Long Short-Term Memory (LSTM) neural networks and NLP to categorize resumes according to skills into various job options. They used total of 962 resumes from Kaggle. Mamta et al. [11] developed an automated resume screening application using Machine Learning (ML) and NLP to categorize resumes. They used different ML models where K-Nearest Neighbors (KNN) found to be best with accuracy 98.44%. Surya et al. [12] used Support Vector Machines (SVM), Logistic Regression, Random Forest and K-Nearest Neighbors (KNN) Machine Learning models for resume ranking, out of all they found SVM as Best one with accuracy 96%. Gopalakrishna and Varadharajan. [13] used Ensemble Voting Classifier consisting variations of Naïve Bayes, Multinomial, Bernoulli, Logistic Regression, SVM, and KNN. Suhas and Manjunath [14] used Named Entity Recognition (NER) combined with word2vec and cosine similarity to extract and analyze resume data. The accuracy obtained was 79.8%. G. Salton and C. Buckley [15] used TF-IDF to find the importance of words in documents. Y. J. Lai et al. [16] proposed the TOPSIS approach for multi-objective decision-making, where alternatives are ranked according to their relative closeness to the ideal best and worst solutions.

The contributions of this work are:

- Identifying new features that can contribute towards resume ranking.
- Labeling resumes through learned and experienced experts.
- Training ML models to categorize the resumes for quality.
- Identify the best ML algorithm for resume quality grading.

II. Methodology

A. Dataset preparation

Dataset was created by collecting resumes from 2025 and 2026 pass out batches of our college. A total of 200 resumes were gathered. Every resume was provided in PDF layout, which introduces challenges inclusive of inconsistent layouts, non-wide spread headings, and unstructured textual content. Therefore, a strong resume parsing and feature extraction pipeline was developed [7], Which extract features such as skills, number of projects, number of certifications, number of GitHub repositories, Quality of GitHub repositories, number of hackathons participated, number of coding problems solved, Professional body membership, CGPA.

Mostly resumes are not always in the same structure, we used NLP methods to extract features from resumes. We used Pdfplumber python library for extracting text and used PyMuPDF python library for extracting links. However, it should be noted that resumes could be of different types, such as scanned or image-based PDF files, and in such cases, the extraction of text information might not be possible. To address this issue, an Optical Character Recognition (OCR) module was implemented, which helps to extract text information from the images of resumes. The skills are identified by using a mix of methods like looking for specific words, understanding synonyms and using Sentence Transformer embeddings [15]. We found certifications and projects by looking for section headings and then used rules to make sure we didn't include the wrong information. One new part of this work is the GitHub repos Score, which we calculated using the GitHub REST API. Remaining features like coding problems solved, LinkedIn, professional body membership, CGPA are extracted using regular expressions [5]. Resume parsing is required as the PDF document contains unstructured text data that cannot be processed by machine learning algorithms. The parsing process converts the resume into structured text data, which allows us to extract important attributes such as skills, number of projects, number of certifications, number of GitHub repositories, Quality of GitHub repositories, number

of hackathons participated, number of coding problems solved, CGPA. The machine learning algorithm cannot identify or quantify important information in the resume document without the parsing process. Finally all features are saved as a csv file and for grading these resumes a team of our college alumni experts and experienced faculty are assigned. They graded all resumes into categories ranging from A to D, and the dataset to train and test the ML algorithms is prepared.

In addition, the process involved several steps in resume preprocessing, including normalization of the text, removal of irrelevant symbols in the resume, and filtering of duplicative pieces of information. Overall, the robustness of the approach is achieved through the combination of both the rule-based technique and the semantic embeddings method. Furthermore, the inclusion of the OCR functionality increases the coverage of the system as it will be capable of processing non-editable resumes, as this is a realistic situation in most recruitment processes. Also, the inclusion of the GitHub profiles in the feature extraction process allows for a broader evaluation of the candidates based on their realistic source code contributions.

Algorithm 1. Feature extraction and grading

Input: pdf resume

- Extract text T and links L from PDF.
- If T is empty → apply OCR.
- Extract skills S using hybrid NLP.
- Count projects P from project section.
- Count certifications C from certification section.
- Extract GitHub username U.
- Fetch repositories and compute GitHub score GS.
- Extract hackathons H and coding problems CP.
- Detect LinkedIn and memberships.
- Extract CGPA value.
- Construct feature vector X.
- Transform skills using TF-IDF

$$TFIDF(t,d) = TF(t,d) \cdot \log\left(\frac{N}{DF(t)}\right)$$

- Predict grade using trained XGBoost classifier.

Output: Return final grade G ∈ {A, B, C, D}.

The algorithm 1 shows the workflow of the proposed automated resume grading system. The system takes a resume PDF as input, from which both text data and embedded hyperlinks are extracted using PDF parsing (and OCR if necessary). The extracted data is then passed through an NLP-based feature extraction module, which extracts important features such as skills, number of projects, number of certifications, number of GitHub repositories, Quality of GitHub repositories, number of hackathons participated, number of coding problems solved, CGPA. TF-IDF processes the Skills text, creating a vocabulary of all skills that exist in the dataset. It then determines the frequency of each skill in all resumes to obtain the document frequency (DF). Based on this, it calculates the inverse document frequency (IDF) to assign more weight to less frequent skills. It then determines the term frequency (TF) of each skill in a particular resume. Finally, TF is multiplied by IDF to obtain the TF-IDF weight, which transforms the skills section into a numerical vector that can be merged with other features number of projects, number of certifications, number of GitHub repositories, Quality of GitHub repositories, number of hackathons participated, number of coding problems solved, CGPA [15]. These features are then standardized and scaled before being passed to a different machine learning classifiers such as Logistic Regression, Decision Tree Classifier, XGBoost Classifier, Linear Sup-port Vector Machine (SVM), K-Nearest Neighbors (KNN) algorithms. Finally, the system provides a predicted resume grade from A to D.

Algorithm 2. GitHub repositories scoring algorithm

- Construct decision matrix for all n repositories using 7 metrics:

$$X = [x_{ij}], i = 1..n, j = 1..7$$

- Normalize criteria values using vector normalization:

$$r_{ij} = \frac{x_{ij}}{\sqrt{\sum_{k=1}^{n} x_{kj}^2}}$$

- Apply weights to get weighted matrix:

$$v_{ij} = w_j \cdot r_{ij}, \sum_{j=1}^{7} w_j = 1$$

- Calculate TOPSIS closeness using ideal best and worst

$$C_i = \frac{D_i^-}{D_i^+ + D_i^-}$$

Where,

$$D_i^+ = \sqrt{\sum_{j=1}^{7} (v_{ij} - A_j^+)^2},\ D_i^- = \sqrt{\sum_{j=1}^{7} (v_{ij} - A_j^-)^2}$$

- Final user score (0–100) as average repo closeness:

$$score(U) = \frac{100}{n}\sum_{i=1}^{n} C_i$$

The algorithm 2 shows the workflow of scoring GitHub repositories, In this TOPSIS technique is employed in our GitHub scoring algorithm to rank the repositories on various engineered quality factors like stars, forks, contributors, pull requests, issue health, freshness, and releases. A decision matrix is formed for each user, where each repository is described by these factors. The TOPSIS technique normalizes the values and assigns weights to the factors based on their relative importance [16]. The best and worst repository solutions are also determined. The distance of each repository from the best and worst solutions is calculated, and a closeness index is derived, which measures the relative closeness of a repository to the best engineered project. The average TOPSIS closeness index of all the repositories is then normalized to get a single GitHub repositories score between 0 and 100 for the user.

B. Machine Learning Models

Logistic Regression: Logistic Regression was used as a basic classifier model to predict the resume grade using the extracted features. It fails to handle the com-plex relations [4]. Decision Tree Classifier: A decision tree classifier uses a tree like structure in order to make decisions and predict resume grades [8]. K-Nearest Neighbors (KNN): KNN compares resume with similar resumes then predict the grade. It is simple to implement but reduces accuracy when features are increased [13]. XGBoost Classifier: XGBoost Classifier based on gradient boosted decision trees, which is used for improve accuracy. Which handles com-plex relations [1]. Linear Support Vector Machine (SVM): Linear SVM grades resumes by separating each class by using hyperplane, which is decision boundary [12].

C. Architecture

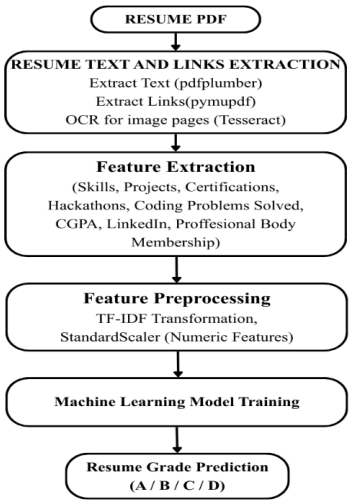


Fig. 1. Architecture Diagram for proposed algorithm

Fig. 1 shows the workflow of proposed system, which follows a multi stage architecture.The process begins by retrieving text and hyperlinks from resumes in PDF format, and Optical Character Recognition is used for image-based resumes. Key attributes such as skills, number of projects, number of certifications, number of GitHub repositories, Quality of GitHub repositories, number of hackathons participated, number of coding problems solved, CGPA are identified and processed using regular text and NLP techniques. The attributes are then input into a trained machine learning model that predicts a grade for each resume based on whether it is an A, B, C, or D grade.

III. RESULTS AND DISCUSSION

Table 1 shows the prime performance measures given by various ML tools that are considered for this study. It is worth mentioning that the XGBoost classifier worked well with an accuracy of 93%, and this is because XGBoost is an ensemble boosting technique, which combines several weak decision trees. This is also because it is effective in handling complex relationships. Which is also reduces overfitting. Therefore, Among the all models XGBoost model is best suitable for this work.

TABLE I. RESUME GRADING ACCURACIES PRODUCED BY VARIOUS ML ALGORITHMS

ML Model	**RESULTS**			
	Precision	***Recall***	***F1 score***	***Accuracy***
Logistic Regression	0.70	0.59	0.62	0.68
Decision Tree	0.79	0.60	0.63	0.72
XG Boost	**0.97**	**0.84**	**0.88**	**0.93**
K-Nearest Neighbors (KNN)	0.54	0.53	0.52	0.68
Linear SVM	0.75	0.66	0.69	0.75

The confusion matrix for the result obtained by XGBoost classifier is given in Fig. 2. This clearly shows that among 40 test resumes, 37 resumes are graded correctly by the XGBoost model with an accuracy of 93%. This model predicts Class A and C with accuracy of 100% but Class D with accuracy 50% and Class B with accuracy 86.

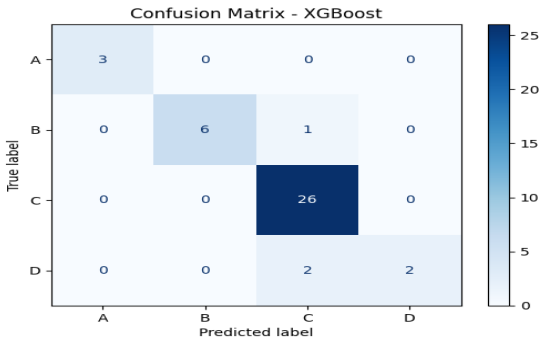


Fig. 2. Confusion Matrix for XGBoost

Fig. 3 gives the graphical comparison of ROC curves of the classifiers performances under evaluation. Fig. 3 clearly shows the performance of XGBoost over others. Among them, XGBoost reaches the highest AUC value, 0.91, thus providing the best performance among all models and a strong capacity to differentiate between positive and negative instances of resume quality. Logistic Regression performs competitively with an AUC of 0.84, while SVM and Decision Tree both have a middle discrimination capability for the two classes with AUC values of 0.85 and 0.74 respectively. In contrast, KNN has the worst AUC value 0.79. Thus, it has the weakest separation ability. Its true positive rate is always higher compared to other models within a wide range of false positive rates, which verifies its robustness and reliability.

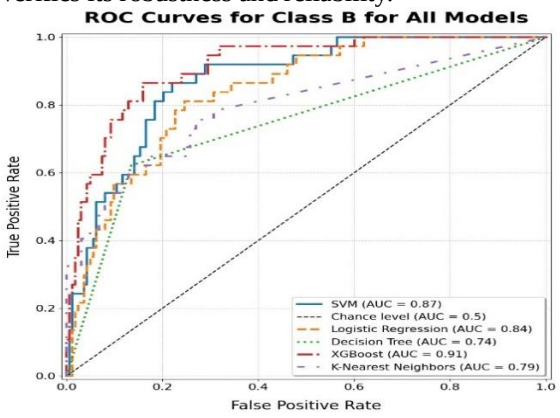


Fig. 3. ROC curves of various classifiers under evaluation

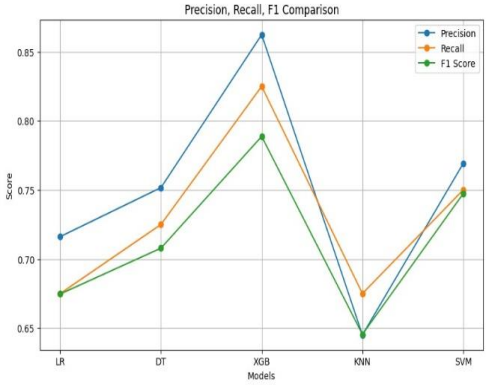


Fig. 4. Precision,Recall,F1 Comparison

As illustrated in Fig. 4, the XGBoost model achieved the highest performance with a precision of approximately 0.86, recall of 0.82, and F1-score of 0.79, indicating its superior capability in accurately classifying resumes. In contrast, KNN exhibited the lowest performance with an F1-score of around 0.64, suggesting limitations in handling complex feature relationships.

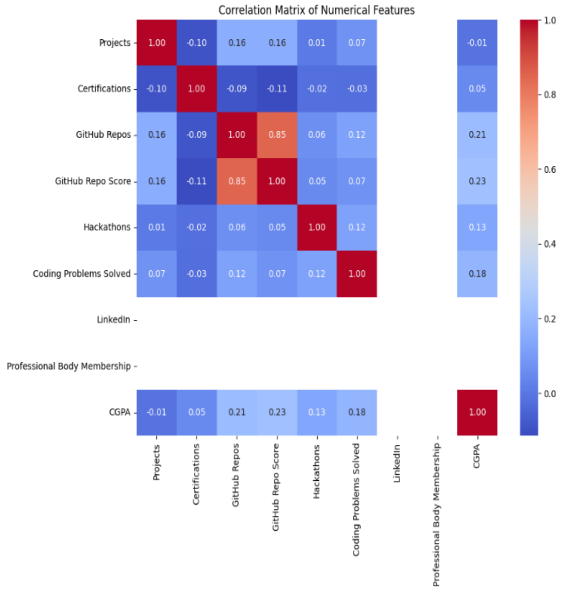


Fig. 5. Correlation matrix of numerical features

Fig. 5 presents the correlation matrix of numerical features used in the proposed system. A strong positive correlation is observed between GitHub repositories and GitHub score (≈0.85), while most other features exhibit weak correlations, indicating minimal redundancy. This diversity of features enhances the model's ability to learn independent patterns and improves classification performance.

IV. Conclusion

In this paper, we developed a resume grading system using Machine Learning (ML) and Natural Language Processing (NLP) methods to assist HR team in Resume grading. A total of 200 resumes were gathered from 2025 and 2026 batches of our college. Features such as skills matching, number of projects, number of certifications, number of GitHub repositories, Quality of GitHub repositories, number of hackathons participated, number of coding problems solved, CGPA are extracted from resumes using NLP methods. These collected resumes are graded by alumni and faculty experts. Logistic Regression, Decision Tree Classifier, XGBoost Classifier, Linear Support Vector Machine (SVM), K-Nearest Neighbors (KNN) algorithms are considered for evaluation. Among the all models XGBoost Classifier is found to be best for this work

with an accuracy of 0.93 and an F1-score of 0.88. Overall, the proposed system reduces the manual work-load of HR team and improve efficiency of resume grading. A fair assessment of the potential and drawbacks of each classifier, based on various performance metrics, is ensured during the comparative evaluation. It can be inferred that the high accuracy of the model achieved by XGBoost Classifier is due to the model's ability to handle non-linear relationships between various heterogeneous features present in the resumes. This system can be extended for larger datasets and different recruiting scenarios; thus, the proposed approach can be used for real-world applications. Future work can extend this work with larger datasets. The system can also be improved by implementing job description-specific ranking to better match candidates with jobs. Additionally, more advanced deep learning models can be explored for better semantic understanding of resumes and more accurate grading.

REFERENCES

[1] A. R. Vishaline, R. K. P. Kumar, S. V. V. N. S. Pramod, K. V. K. Vignesh, and P. Sudheesh, "An ML-based resume screening and ranking system," in Proc. 2024 Int. Conf. Signal Processing, Computation, Electronics, Power and Telecommunication (IConSCEPT), Karaikal, India, 2024, pp. 1–6.

[2] K. Tejaswini, V. Umadevi, S. M. Kadiwal, and S. Revanna, "Design and development of machine learning based resume ranking system," Global Transitions Proc., vol. 3, no. 2, pp. 371–375, 2022.

[3] C. A. Reddy, B. S. Jayanth, B. Rithvik, B. Avinash, and R. Tiwari, "Intelligent resume tracking system," J. Technol., vol. 12, no. 4, pp. 697–705, 2024.

[4] P. K. Roy, S. S. Chowdhary, and R. Bhatia, "A machine learning approach for automation of resume recommendation system," in Proc. Int. Conf. Computational Intelligence and Data Science (ICCIDS), Noida, India, 2019, pp. 2318–2327.

[5] D. Chennupalle, T. Gaddam, and C. R. Kavitha, "Intelligent resume parser and classifier using regular expressions," in Proc. 4th Int. Conf. Smart Technologies, Communication and Robotics (STCR), Sathyamangalam, India, 2025, pp. 1–6.

[6] S. Tayal, T. Sharma, S. Singhal, and A. Thakur, "Resume screening using machine learning," Int. J. Sci. Res. Comput. Sci. Eng., vol. 10, no. 2, pp. 602–606, 2024.

[7] P. D. Lakshmi, C. Vishnuvardhan, G. Rajeev, and K. N. S. K. Kumar, "Automated resume screening using natural language processing," J. Emerging Technol. Innovative Res., vol. 10, no. 3, pp. 100–104, 2023.

[8] B. Surendiran, T. Paturu, H. V. Chirumamilla, and M. N. R. R. Reddy, "Resume classification using ML techniques," in Proc. 2023 Int. Conf. Signal Processing, Computation, Electronics, Power and Telecommunication (IConSCEPT), Karaikal, India, 2023, pp. 1–5.

[9] C. G. Onukwugha, C. I. Ofoegbu, O. B. Aliche, and C. U. Betrand, "Resume optimization model using machine learning," Int. J. Intell. Inf. Syst., vol. 13, no. 5, pp. 109–116, 2024.

[10] S. Bharadwaj, R. Varun, P. S. Aditya, M. Nikhil, and G. C. Babu, "Resume screening using NLP and LSTM," in Proc. Int. Conf. Inventive Computation Technologies (ICICT), Nepal, 2022, pp. 238–241.

[11] M. Mamta, A. K. Jha, D. Bhandari, and Ankit, "Resume screening app using machine learning," Int. J. Sci. Res. Develop., vol. 12, no. 9, pp. 50–52, 2024.

[12] C. H. Surya, G. Kiriti, K. S. Saketh, P. S. Charan, and A. Anjali, "Optimizing job matching through automated resume screening with NLP and machine learning," in Proc. 2024 IEEE Int. Conf. Intelligent Signal Processing and Effective Communication Technologies (INSPECT), Gwalior, India, 2024, pp. 1–6.

[13] S. T. Gopalakrishna and V. Varadharajan, "Automated tool for resume classification using semantic analysis," Int. J. Artif. Intell. Appl., vol. 10, no. 1, pp. 11–23, 2019.

[14] S. He and M. Ae, "Differential hiring using a combination of NER and word embedding," Int. J. Recent Technol. Eng., vol. 9, no. 1, pp. 50–56, 2020.

[15] G. Salton and C. Buckley, "Term-weighting approaches in automatic text retrieval," Inf. Process. Manage., vol. 24, no. 5, pp. 513–523, 1988.

[16] Y. J. Lai, T. Y. Liu, and C. L. Hwang, "TOPSIS for MODM," Eur. J. Oper. Res., vol. 76, no. 3, pp. 486–500, 1994.